

# Data Lab 实验报告

位级运算与浮点数

## 一、完成情况

这次在 `bits.c` 中实现了全部 19 个函数。本地运行 `./btest`，每项的 Errors 都是 0，总分 110/110；GitHub Actions 的自动测试也得到 110/110。

## 二、测试记录

|         |                |                                                                                                        |
|---------|----------------|--------------------------------------------------------------------------------------------------------|
| Status  | Total duration |  Autograding complete |
| Success | <u>17s</u>     | Points 110/110                                                                                         |

图 1 GitHub Actions 运行记录：状态 Success，总分 110/110。[查看原始记录](#)。

本地功能测试：下面是我运行 `./btest` 后的输出，按原结果重新排了版，不是终端原生截图。

```
$ ./btest
Score Rating Errors Function
1 1 0 signMask
2 2 0 bitXor
3 3 0 negativePart
4 4 0 copyByteWithin
4 4 0 logicalShift
4 4 0 swapNibblePairs
4 4 0 secondLowestZeroBit
5 5 0 oddParity
5 5 0 rotateRightBits
5 5 0 roundEvenPow2
5 5 0 midpointTowardFirst
7 7 0 isBetweenEitherOrder
7 7 0 mul5Sat
7 7 0 classifyAdd3
7 7 0 floatScaleThreeHalves
10 10 0 floatRoundEven
10 10 0 float_i2f
10 10 0 bitCount
10 10 0 bitReverse
Total points: 110/110
```

### 三、各函数的实现思路

#### P01 signMask

直接把 1 左移 31 位，最高位为 1，其余位为 0。

#### P02 bitXor

异或在两位不同时才为 1。我把它拆成“至少有一个 1”和“不能两个都是 1”，这样只需 ~ 和 &。

#### P03 negativePart

先用  $\sim x + 1$  求相反数。 $x >> 31$  在负数时全为 1，可以直接用它作选择掩码。

#### P04 copyByteWithin

字节编号乘 8 就是位移量。取出源字节后，清掉目标字节，再把源字节放进去。

#### P05 logicalShift

C 的有符号右移会补符号位，所以右移后还要用掩码清掉左边补进来的 1。

#### P06 swapNibblePairs

用重复的 0x0f 掩码分出每个字节的高、低半字节，分别右移和左移 4 位后合并。

#### P07 secondLowestZeroBit

先取反，把要找的 0 变成 1；去掉最低的那个 1，再取剩余位中最低的 1。

#### P08 oddParity

连续与右移后的自身异或，把 32 位的奇偶性逐步折到最低位；最低位为 0 说明原数有偶数个 1。

#### P09 rotateRightBits

先把  $n$  限制在 0 到 31。右移部分清除符号扩展位，左移部分补回绕到高位的那些比特。

#### P10 roundEvenPow2

只看除以 2 的  $n$  次方后的余数。大于一半就进位；正好一半时，看当前倍数是奇数还是偶数。

#### P11 midpointTowardFirst

用  $(x \& y) + ((x \wedge y) >> 1)$  避开  $x + y$  的溢出。若中点恰在两个整数之间，就按  $x$  与  $y$  的大小决定是否加 1。

#### P12 isBetweenEitherOrder

分别判断  $x$  是否不小于  $a$ 、 $b$ 。为了避开减法溢出，先看符号是否相同；两个比较结果不同时， $x$  在端点之间，端点相等另行处理。

#### P13 mul5Sat

把五倍写成  $x + (x \ll 2)$ 。左移和加法都可能溢出，因此两处都检查；溢出后按  $x$  的符号返回上下界。

#### P14 classifyAdd3

分两次做加法，每次根据输入和结果的符号记录溢出方向。把两个方向合起来，就能判断三个数的真实和是否越界。

#### P15 floatScaleThreeHalves

先拆出符号、阶码、尾数。NaN 和无穷大不动；其他值把有效尾数乘 3 后除以 2，并根据丢掉的位做偶数舍入。

#### P16 floatRoundEven

先按阶码判断是否已经是整数。需要舍入时，比较小数部分和 0.5；刚好一半就看保留整数的最低位，舍到偶数。

#### P17 float\_i2f

先用无符号运算取得绝对值，这样 INT\_MIN 也能处理。找到最高的 1 后确定阶码，尾数放不下时按丢弃位舍入。

#### P18 bitCount

按 1、2、4、8、16 位逐级合并计数。每一级用掩码隔开相邻组，避免求和时互相干扰。

#### P19 bitReverse

按 1、2、4、8、16 位大小依次交换相邻分组；掩码从较大的掩码推出来，节省运算符个数。